

会議室予約管理システム システム概要書

アーキテクチャ設計・機能一覧・セキュリティ制御仕様

作成日: 2026年8月28日 Ver 1.0

開発: Windows 11 / VS Code | 本番: Ubuntu Linux 24.04 LTS (Apache Proxy)

1. システム基本情報 & 目的

システム名	会議室予約管理システム
利用対象	一般社内ユーザー、システム管理者
目的	Web経由でのリアルタイム会議室空き状況照会、予約申請、および管理者による統括管理。
認証方式	Session / Cookie 認証 (ロール別アクセス制御)

2. 開発・動作環境

区分	環境仕様 / 技術要素
開発環境	Windows 11, VS Code, Node.js
本番OS	Ubuntu Linux 24.04 LTS
Web/Proxy	Apache HTTP Server (ProxyPass)
App Server	Node.js (Express) + Systemd daemon
Database	PostgreSQL 16 (node-postgres / pg)

3. データベース構成 (PostgreSQL)

テーブル	主要カラム	役割
users	id, username, password_hash, role	ユーザー情報・権限管理
rooms	id, room_name, capacity	会議室マスタ
bookings	id, room_id, user_id, booking_date, start_time, end_time	予約データ (時間重複制限)

4. システムアーキテクチャ

```
[ Client (Browser) ] | HTTP / HTTPS (Port 80 / 443)
▼ [ Apache Web Server ] (Ubuntu 24.04) | ProxyPass
http://localhost:3000/ ▼ [ Express Node.js App ]
(Port 3000) | 認証ミドルウェア (checkAuth) | テンプレートエンジン (EJS) | DBクライアント (pg) | ▼ [ PostgreSQL DB ] (Port 5432)
```

※ Apacheを前段の逆プロキシとし、Node.jsアプリをSystemdでサービス化して稼働。

5. 認証ミドルウェア (`checkAuth`) 仕様

権限に応じたアクセス制御と `next()` バトンタッチ処理:

```
function checkAuth(role) {
  return function(req, res, next) {
    if (!req.session.user) return
    res.redirect('/');
    if (role === 'admin' && req.session.user !== 'admin') {
      return res.status(403).send('閲覧権限がありません');
    }
    next(); // チェック合格: 次のルーティング処理へ移行
  };
}
```

※ `checkAuth('admin')` が「関数を返す関数」として動作し、Expressのガード関門を構築。

6. 主要機能 & 権限マトリクス

機能	一般	管理	概要
ログイン/出	○	○	セッション確立・破棄
予約参照	○	○	日時・時間別の一覧表示
新規予約	○	○	時間重複を防止して登録
予約取消	自件	全件	登録済み予約の削除
管理画面	×	○	全社データ統括・利用分析

7. エンドポイント構成 (Routing)

Route	Middleware	概要
`GET /`	なし	トップ / ログイン画面
`GET /bookings`	`checkAuth('user')`	予約画面表示
`GET /admin`	`checkAuth('admin')`	管理者専用ダッシュボード

8. データ整形・プログラミング規約

- 時間整形 (`formatTime`):** DBの `HH:mm:ss` を `HH:mm` (5文字) に切り出し整形。
- 日付整形 (`formatDate`):** `YYYY-MM-DD` 形式の共通整形処理。
- 不変性の確保 (`const`):** 変数・関数の意図せぬ再代入を防ぎ、安全性を担保。